CURSO 1 · AULA 5

Ordem, Idempotência e Semânticas de Entrega



at-most-once · at-least-once · exactly-once — e o trio de confiabilidade

Aula gravada · ~20 min · inclui demo (sandbox Docker)

O que vamos ver

01 Ordem por chave — a anatomia do roteamento

02 A escrita replicada: líder, ISR e acks

03 O dilema da entrega: as 3 semânticas

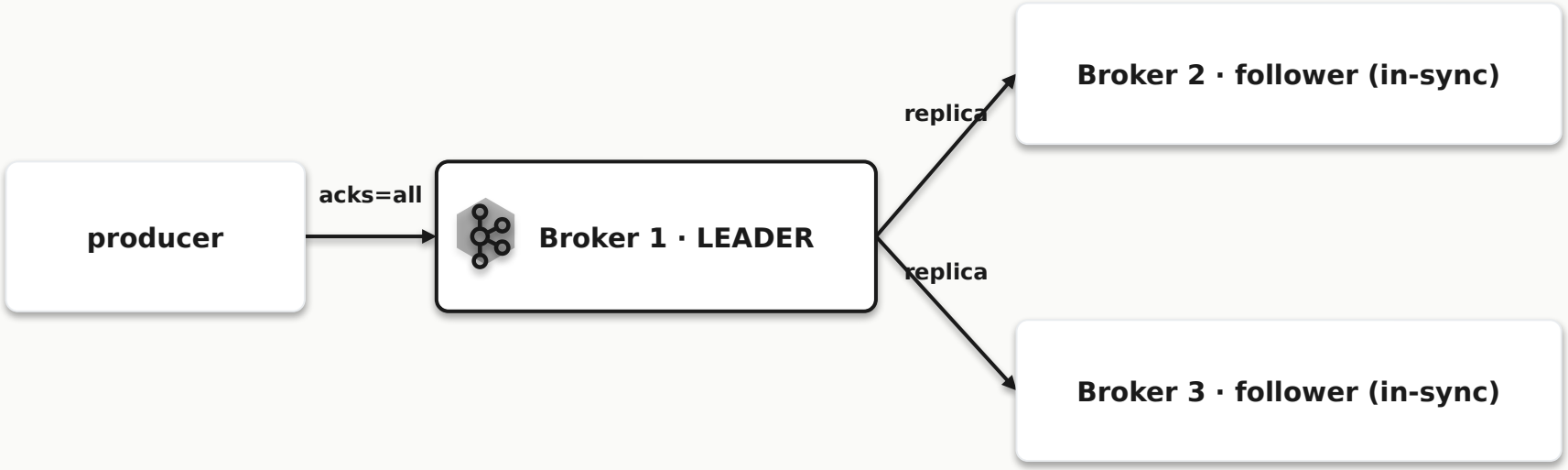
04 Idempotência: PID + sequence (e o consumidor)

05 Demo: ordem, duplicata e a duplicata inofensiva

Ordem por chave: o roteamento



A escrita replicada: líder, ISR e acks



O dilema da entrega: 3 semânticas

At-most-once

Não reenvia

Pode PERDER

Custo mínimo

Telemetria tolerante

At-least-once

Reenvia até confirmar

Pode DUPLICAR

O padrão de mercado

+ idempotência = ouro

Exactly-once

Reenvia + deduplica

Não perde, não duplica

Custo mais alto

Movimentação financeira

Idempotência no producer: PID + sequence

Cada producer ganha um PID e numera os records por partição. Reenvio com sequência repetida? O broker descarta em silêncio.



- enable.idempotence=true — DEFAULT nas versões atuais
- O ack se perdeu na rede? Reenvia sem medo — não duplica na escrita
- É por isso que retries alto é seguro (a tabela da Aula 4)

Idempotência no consumidor: a síntese

A duplicata de RELEITURA (rebalance, restart) só o design do consumidor resolve — o broker não adivinha o efeito de negócio.



- Técnicas: chave natural + upsert · 'já processei este event_id?'
- at-least-once + consumidor idempotente ≈ efeito exactly-once
- EOS completo (transações) existe — com custo e escopo

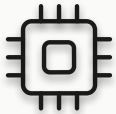
Demo: ordem, duplicata, e a duplicata ~~inofe~~nsiva

Ciclo de vida de 3 pedidos intercalados, key = order_id. Cenas:

```
# Cena 1 — produzir COM chave → eventos do MESMO pedido:
#   mesma partição, na ordem de negócio (contrastar c/ Aula 3)
# Cena 2 — consumer com commit manual: processa e MORRE antes
#   de commitar → religa → a DUPLICATA aparece na tela
# Cena 3 — consumidor idempotente (dedup por event_id):
#   a releitura acontece igual — o EFEITO não se repete
```

Esperado: Ordem onde o negócio pede · perda nunca · duplicata neutralizada. Não é bug do Kafka — é o contrato da semântica.

PONTO DE REFLEXÃO



RF=3, mas o producer usa acks=1. O líder confirma e morre antes de replicar. O dado existe?

Pause. E o que mudaria com acks=all + min.insync.replicas=2?

Recap

- Mesma chave → mesma partição → ordem de negócio
- O trio: RF=3 + min.insync.replicas=2 + acks=all — juntos
- 3 semânticas: perder × duplicar × pagar pelo exato
- PID+seq: retry sem duplicar NA ESCRITA
- Consumidor idempotente: a duplicata de releitura, neutralizada

Pipeline confiável = ordem declarada + trio de replicação + duplicata inofensiva.

A seguir → Aula 6 — Do Monólito ao EDA: quando migrar